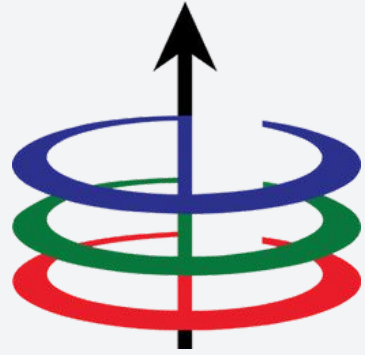




Lecture 2.2

Proving Algorithm Correctness

EEE 121 THVW1

Steven S. Sison

- Introduction
- Proof by Induction
- Loop Invariants

- **Introduction**
- Proof by Induction
- Loop Invariants

Mabilis nga, pero tama ba?

In our previous lecture, we learned about how to determine the time complexity of our code.

- This answers the question about how efficient our algorithm is in terms of time.

Mabilis nga, pero tama ba?

In our previous lecture, we learned about how to determine the time complexity of our code.

- This answers the question about how efficient our algorithm is in terms of time.

But what about the correctness of our algorithm?

Mabilis nga, pero tama ba?

In our previous lecture, we learned about how to determine the time complexity of our code.

- This answers the question about how efficient our algorithm is in terms of time.

But what about the correctness of our algorithm?

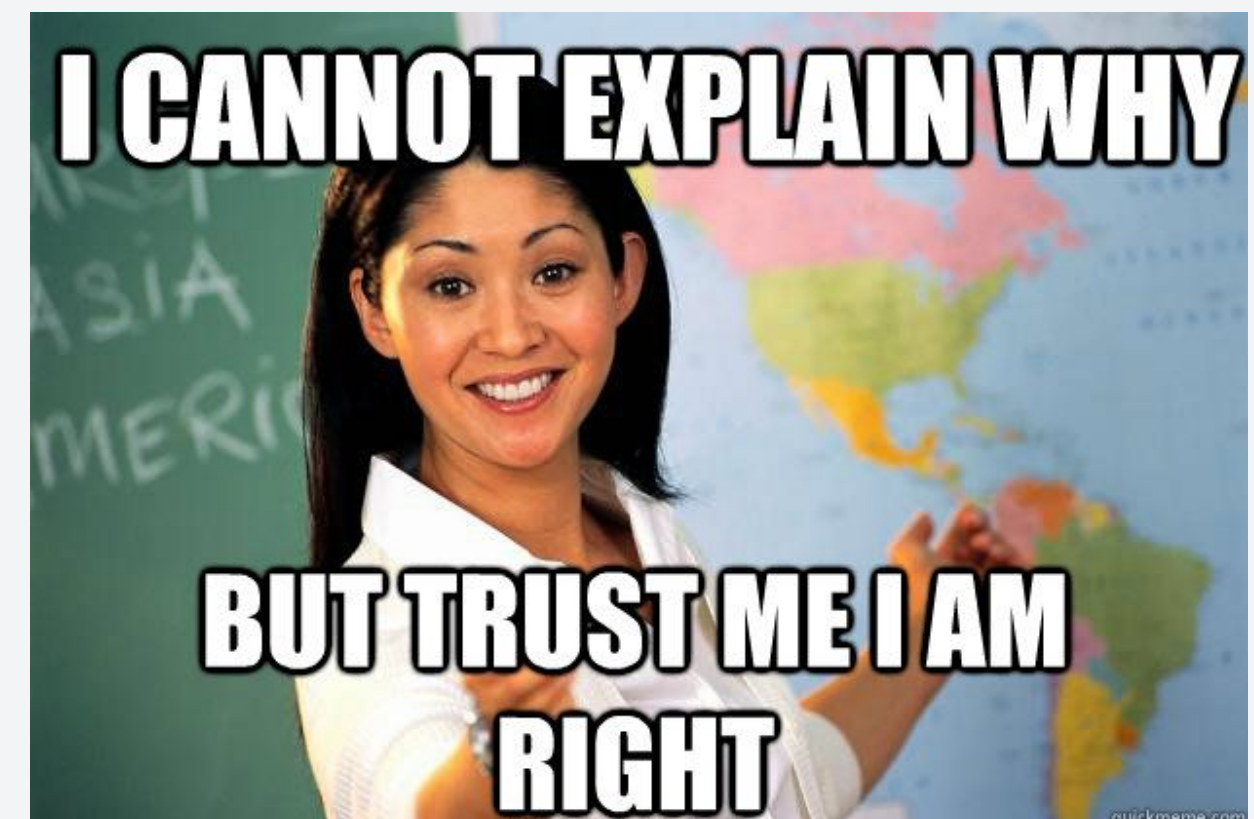


Mabilis nga, pero tama ba?

In our previous lecture, we learned about how to determine the time complexity of our code.

- This answers the question about how efficient our algorithm is in terms of time.

But what about the correctness of our algorithm?



Mabilis nga, pero tama ba?

In our previous lecture, we learned about how to determine the time complexity of our code.

- This answers the question about how efficient our algorithm is in terms of time.

But what about the correctness of our algorithm?



Mabilis nga, pero tama ba?

In our previous lecture, we learned about how to determine the time complexity of our code.

- This answers the question about how efficient our algorithm is in terms of time.

But what about the correctness of our algorithm?



Proving algorithm correctness

The fool-proof way to prove an algorithm is to exhaust all possible inputs and ensure that the corresponding output is correct.

- **THIS IS IMPOSSIBLE!**

We have to use mathematical techniques to back-up our claim

There are two common approaches to do this:

1. Proof by Induction
2. Loop invariants

- Introduction
- **Proof by Induction**
- Loop Invariants

Proof by Induction

When using proof by induction, the aim is to prove that a statement $P(n)$ is true for all positive integers, starting with $n = 1$ (**base case**). To do this, you need to perform the following steps:

1. Prove the base case
 - a. You need to prove that $P(n)$ is true for $n = 1$.
2. Prove the inductive step
 - a. Assume that the statement $P(n)$ is true for an arbitrary value of n .
 - b. Using $P(n)$, prove that the statement $P(n+1)$ is also true (**inductive step**).
3. Show that the function terminates.

Magulo ba? Puro math? Gawan natin ng example :)

Example 1: Factorial

Consider the code below that returns the value of an exponential expression:

```
int factorial(int num){  
    if (num == 0){  
        return 1;  
    }  
    int result = 1;  
    for (int i = 1; i <= n; i++){  
        result *= i;  
    }  
    return result;  
}
```

Base Case:

- Since $\text{factorial}(0) = 1 = 0!$, the base case is true.

Example 1: Factorial

Consider the code below that returns the value of an exponential expression:

```
int factorial(int num){
    if (num == 0){
        return 1;
    }
    int result = 1;
    for (int i = 1; i <= n; i++){
        result *= i;
    }
    return result;
}
```

Base Case:

- Since $\text{factorial}(0) = 1 = 0!$, the base case is true.

Inductive Step

- Assume $\text{factorial}(k)$ is correct.
- If we run $\text{factorial}(k+1)$, we know that at $i = k$, $\text{result} = k!$ (by our inductive hypothesis).
- At $i = k+1$, $\text{result} = k! \times (k+1) = (k+1)!$

Example 1: Factorial

Consider the code below that returns the value of an exponential expression:

```
int factorial(int num){
    if (num == 0){
        return 1;
    }
    int result = 1;
    for (int i = 1; i <= n; i++){
        result *= i;
    }
    return result;
}
```

Base Case:

- Since $\text{factorial}(0) = 1 = 0!$, the base case is true.

Inductive Step

- Assume $\text{factorial}(k)$ is correct.
- If we run $\text{factorial}(k+1)$, we know that at $i = k$, $\text{result} = k!$ (by our inductive hypothesis).
- At $i = k+1$, $\text{result} = k! \times (k+1) = (k+1)!$

Based on the code, the loop terminates at n .

By induction, the code is correct.

Example 2: Solving for the power

Consider the code below. Let's prove the claim that $\text{power}(m,n) = m^n$ for $m,n \geq 0$.

```
int power(int m, int n) {  
    if(n == 0)  
        return 1;  
  
    p = power(m, n/2);  
  
    if(n%2 == 0)  
        return p*p;  
    else  
        return p*p*m;  
}
```

Example 2: Solving for the power

Consider the code below. Let's prove the claim that $\text{power}(m,n) = m^n$ for $m,n \geq 0$.

Base Case:

```
int power(int m, int n) {  
    if(n == 0)  
        return 1;  
  
    p = power(m, n/2);  
  
    if(n%2 == 0)  
        return p*p;  
    else  
        return p*p*m;  
}
```

- At $n = 0$, $\text{power}(m,0) = 1 = m^0$.
- Thus, the base case is true.

Example 2: Solving for the power

Consider the code below. Let's prove the claim that $\text{power}(m,n) = m^n$ for $m,n \geq 0$.

```
int power(int m, int n) {  
    if(n == 0)  
        return 1;
```

Base Case:

- At $n = 0$, $\text{power}(m,0) = 1 = m^0$.
- Thus, the base case is true.

Inductive Step

Inductive step doesn't always have to be n and $n+1$

- We assume that $\text{power}(m, n/2) = m^{n/2}$ 😊 (inductive step)
- If $n\%2 = 0$, $n = 2k$.
 - $\text{power}(m,n) = (m^k)(m^k) = m^{2k} = m^n$
- If $n\%2 = 1$, $n = 2k+1$.
 - $\text{power}(m,n) = (m^k)(m^k)(m^1) = m^{2k+1} = m^n$

```
    p = power(m, n/2);  
  
    if(n%2 == 0)  
        return p*p;  
    else  
        return p*p*m;
```

```
}
```


Example 2: Solving for the power

Consider the code below. Let's prove the claim that $\text{power}(m,n) = m^n$ for $m,n \geq 0$.

Inductive Step

```
int power(int m, int n) {  
    if(n == 0)  
        return 1;  
  
    p = power(m, n/2);  
  
    if(n%2 == 0)  
        return p*p;  
    else  
        return p*p*m;  
}
```

- We assume that $\text{power}(m, n/2) = m^{n/2}$ (inductive step)
- If $n\%2 = 0$, $n = 2k$.
 - $\text{power}(m,n) = (m^k)(m^k) = m^{2k} = m^n$
- If $n\%2 = 1$, $n = 2k+1$.
 - $\text{power}(m,n) = (m^k)(m^k)(m^1) = m^{2k+1} = m^n$

Since n is decreasing and code has a base case, the function terminates.

By induction, the code is correct.

Proof by Strong Induction

The standard proof by induction works under these three conditions.

1. Prove the base case, $P(1)$.
2. Assume $P(n)$ is true and prove the inductive step $P(n+1)$

Proof by **strong** induction works by modifying the conditions of the standard proof by induction.

- Instead of just proving the base case, $P(1)$, we have to prove $P(1), P(2), \dots, P(n)$.

- Introduction
- Proof by Induction
- **Loop Invariants**

The Horse Paradox

Let's say we have a lot of horses. We want to prove the claim that all the horses we have have the same color.

- 1. Base Case:** Horse 1 has the same color (Only one horse so will always be true)
- 2. Inductive Step:** Assume n horses has the same color.
- 3. Conclusion.** All horses have the same color.

The Horse Paradox

Let's say we have a lot of horses. We want to prove the claim that all the horses we have have the same color.

- 1. Base Case:** Horse 1 has the same color as horse 1 (true for any horse so will always be true)
- 2. Inductive Step:** Assume horse 1 has the same color as horse 2.
- 3. Conclusion.** All horses have the same color.



Just because you only saw same-colored horses, you would conclude that all horses have the same color.

(Baka colorblind ka lang)

Loop Invariants

When functions perform loops, they tend to modify data.

- Performing inductive reasoning sometimes fail in this context.

A **loop invariant** is a part of a code that remains true before, during, and after loop iterations.

Proving using loop invariants builds upon the idea of inductive reasoning.

- Instead of using properties of the code, we prove that the loop invariants are true even after the iterations.

How to prove using loop invariants

To prove using loop invariants, we generally follow these steps:

1. Identify the goal of the loop. This helps us determine a loop invariant.
2. Define the loop invariant. This should be a part of the code that remains true throughout the loop.
3. Prove that the invariant is true during:
 - a. Initialization (before the loop starts)
 - b. Maintenance (during the loop)
 - c. Termination (after the loop)

There's no general guide on what "loop invariant" to choose, it takes skills and practice so don't be discouraged on your first tries 😊

Example 1: Revisiting the factorial function

Recall our factorial function earlier. Previously, we proved its correctness mostly by its mathematical definition. Now, we'll look at a different perspective.

```
int factorial(int n){  
    int result = 1;  
    if (n == 0){  
        return 1;  
    }  
    for (int i = 1; i <= n; i++){  
        result *= i;  
    }  
    return result;  
}
```

Loop Invariant: At the start of any iteration, the variable result contains the product factorial of integer i.

Example 1: Revisiting the factorial function

Recall our factorial function earlier. Previously, we proved its correctness mostly by its mathematical definition. Now, we'll look at a different perspective.

```
int factorial(int n){
    int result = 1;
    if (n == 0){
        return 1;
    }
    for (int i = 1; i <= n; i++){
        result *= i;
    }
    return result;
}
```

Loop Invariant: At the start of any iteration, the variable result contains the product factorial of integer i.

Initialization: Before the first iteration, result = 1 is correct since $1! = 1$.

Example 1: Revisiting the factorial function

Recall our factorial function earlier. Previously, we proved its correctness mostly by its mathematical definition. Now, we'll look at a different perspective.

```
int factorial(int n){  
    int result = 1;  
    if (n == 0){  
        return 1;  
    }  
    for (int i = 1; i <= n; i++){  
        result *= i;  
    }  
    return result;  
}
```

Loop Invariant: At the start of any iteration, the variable result contains the product factorial of integer i.

Initialization: Before the first iteration, $\text{result} = 1$ is correct since $1! = 1$.

Maintenance: Assume that the loop invariant is true at iteration i. That is, $\text{result} = i!$ After iteration i, we know that $\text{result} = (i!)(i+1) = (i+1)!$ Thus, this holds true.

Example 1: Revisiting the factorial function

Recall our factorial function earlier. Previously, we proved its correctness mostly by its mathematical definition. Now, we'll look at a different perspective.

```
int factorial(int n){
    int result = 1;
    if (n == 0){
        return 1;
    }
    for (int i = 1; i <= n; i++){
        result *= i;
    }
    return result;
}
```

Loop Invariant: At the start of any iteration, the variable result contains the product factorial of integer i.

Initialization: Before the first iteration, $\text{result} = 1$ is correct since $1! = 1$.

Maintenance: Assume that the loop invariant is true at iteration i. That is, $\text{result} = i!$. After iteration i, we know that $\text{result} = (i!)(i+1) = (i+1)!$. Thus, this holds true.

Termination: The loop terminates at n, where result contains the product of 1 to n which is $n!$.

Example 2: Getting the sum of an array

Consider a function that sums the n elements of an array.

```
int get_sum(const int* arr,int n) {  
    int total = 0;  
    for(int i = 0; i < n; i++)  
        total += arr[i];  
    return total;  
}
```

Loop Invariant: At the start of iteration i , the variable `total` holds the sum to the elements of `arr[0:i]`. Let's label it as S_i .

Initialization: At $i = 0$, `total = 0` since the sum of an empty array `arr[0:0]` should be zero.

Example 2: Getting the sum of an array

Consider a function that sums the n elements of an array.

```
int get_sum(const int* arr,int n) {  
    int total = 0;  
    for(int i = 0; i < n; i++)  
        total += arr[i];  
    return total;  
}
```

Loop Invariant: At the start of iteration i , the variable `total` holds the sum to the elements of `arr[0:i]`. Let's label it as S_i .

Maintenance: Assume that the loop invariant holds at iteration i , $\text{total} = \text{sum}(\text{arr}[0:i]) = S_i$. After iteration i , $\text{total} = \text{sum}(\text{arr}[0:i]) + \text{arr}[i+1] = \text{sum}(\text{arr}[0:i+1]) = S_{i+1}$.

Example 2: Getting the sum of an array

Consider a function that sums the n elements of an array.

```
int get_sum(const int* arr,int n) {  
    int total = 0;  
    for(int i = 0; i < n; i++)  
        total += arr[i];  
    return total;  
}
```

Loop Invariant: At the start of iteration i , the variable `total` holds the sum to the elements of `arr[0:i]`. Let's label it as S_i .

Termination: The loop terminates at $i = n$. At this point, we know that `total` contains the sum of elements `arr[0]` to `arr[n-1]`, which is also equal to $S_n = \text{sum}(\text{arr}[0:n])$.

Example 3: Something more complicated

Consider the program below. Prove that $y = c$ after the loop.

```
x = c; y = 0;  
while(x > 0) {  
  x--;  
  y++;  
}
```

Any ideas on a possible loop invariant?

Example 3: Something more complicated

Consider the program below. Prove that $y = c$ after the loop.

```
x = c; y = 0;  
while(x > 0) {  
    x--;  
    y++;  
}
```

Loop Invariant: Let's choose $x+y = c$ as the loop invariant. Why? The value of x and y changes as the loop progresses but their sum remains unchanged.

Initialization: Before the loop starts, $x = c$ and $y = 0$. Thus, $x+y = c$ holds.

Example 3: Something more complicated

Consider the program below. Prove that $y = c$ after the loop.

```
x = c; y = 0;  
while(x > 0) {  
    x--;  
    y++;  
}
```

Loop Invariant: Let's choose $x+y = c$ as the loop invariant. Why? The value of x and y changes as the loop progresses but their sum remains unchanged.

Maintenance: At iteration 1, $x = c - 1$ and $y = 1$. At iteration 2, $x = c - 2$ and $y = 2$. Thus, at iteration k , $x = c - k$ and $y = k$. Since $x + y = (c-k) + k = c$, the loop invariant still holds. At the $(k+1)$ th iteration, $x = c - (k+1)$ and $y = k+1$. Thus, $x+y$ is still equal to c .

Example 3: Something more complicated

Consider the program below. Prove that $y = c$ after the loop.

```
x = c; y = 0;  
while(x > 0) {  
    x--;  
    y++;  
}
```

Loop Invariant: Let's choose $x+y = c$ as the loop invariant. Why? The value of x and y changes as the loop progresses but their sum remains unchanged.

Termination: At the final iteration where $x = 0$, this would mean that $x = c - c$ and $y = c$. Thus, $x+y$ is still c . The loop invariant still holds.

Is loop invariant better than induction?

Short answer, no. Much like data structures, there is no perfect technique to prove algorithms.

Proof by Induction

- Typically used in proving recursive functions and properties of a structure.
- Best for proving mathematical theorems.

Loop Invariants

- Focused on proving loop behavior and correctness of iterative algorithms (loop invariant nga eh diba 😊)

Use what is most appropriate for the function or code that you want to prove!

Any questions?

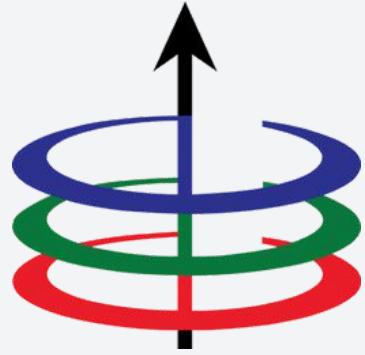
Quiz ulit!

Isang item lang naman ezpz sainyo to (sana pls pls)

Ipaglaban mo na tama!

Given the code below, prove that it returns the maximum element in an array.

```
int find_max(int arr[], int n) {  
    int max_value = arr[0];  
  
    for (int i = 1; i < n; i++) {  
        if (arr[i] > max_value) {  
            max_value = arr[i];  
        }  
    }  
  
    return max_value;  
}
```



Lecture 2.2

Proving Algorithm Correctness

EEE 121 THVW1

Steven S. Sison